CSC 3320: System-Level Programming

Chapter 9 (contd.)

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Chapter 9: Functions (contd.)

Function Declaration

return-type ***function-name*** (*parameters*) ;

```
double average(double x, double y);
```

Declaration

Or you can skip the name of parameters in declaration, and just write
double average (double, double)

```
int main()
{
    ……….
    average(x, y);
}
```

Function call

```
double average(double x, double y)
{
        ……….
}
```

Definition

Arguments vs. Parameters

- *Parameters*
  - Appear in function ***definitions***
  - They are dummy names that represent values to be supplied when the function is called
- *Arguments*
  - Expressions that appear in **function *calls***.

Array Arguments

- Arrays are often used as arguments

```
int f(int a[])
{
    ……
}
```

- How to get the length of the array argument in function?

```
int f(int a[])
{
    int len = sizeof(a)/sizeof(a[0]); /** WRONG**/
}
```

Array Arguments

- Unlike Java, C does not know how long the array is?
- So we will have to supply the length (if needed in function) as an additional arguments.

```
int f(int a[], int n)
{
    ……
}
```

Program Termination

- The *return* statement
  - Using return in ***main*** is one way to exit the program

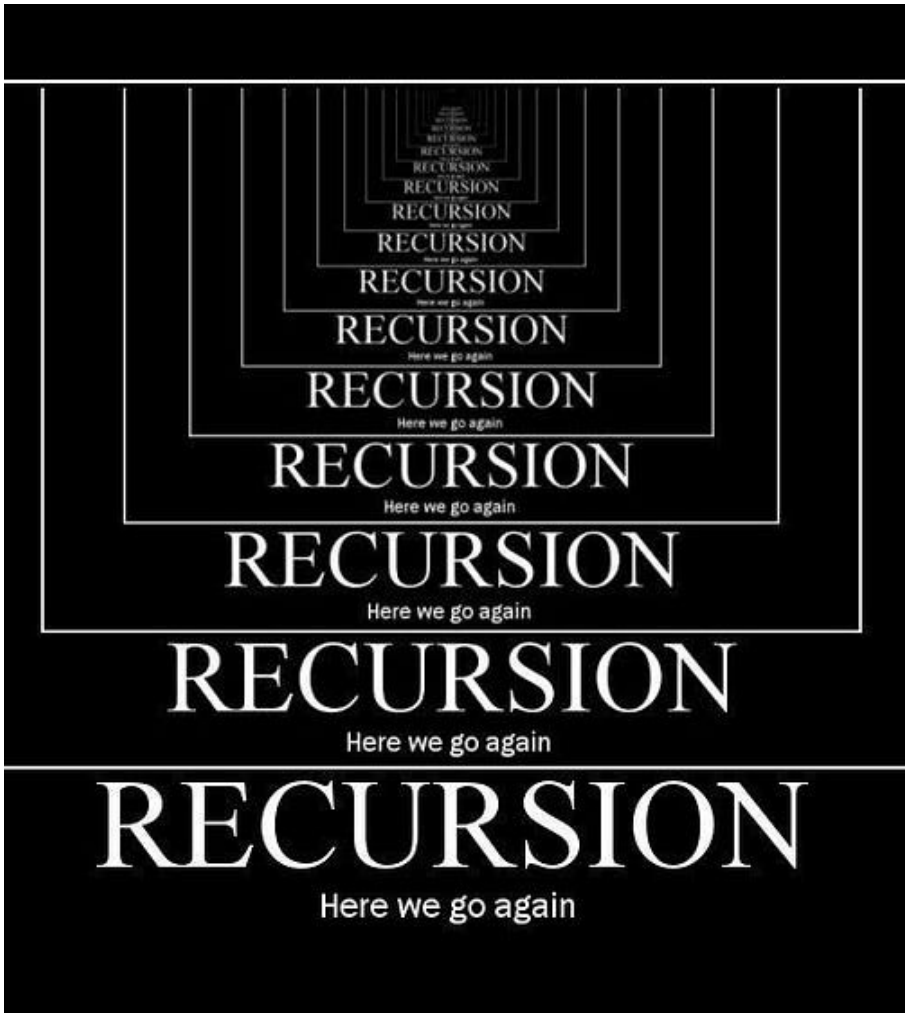
```
return  expression ;
```

- The *exit* function
  - Calling *exit* in any functions is another way to terminate a program.
  - *exit* belongs to <*stdlib.h*>

```
exit(expression) ;
```

```
exit(0);
exit(EXIT_SUCCESS);
exit(EXIT_FAILURE);
```

Recursion



- **Recursion** is a technique where a function **calls itself** to solve a smaller version of the same problem. (**recursive case**)
- Each recursive function needs a **base case** to stop calling itself.

Counting a Russian Nesting Doll

```
int count_dolls(this_doll)
    answer = this_doll + all dolls inside the outer doll
    return answer
/* counting involves opening it and seeing if there is
any smaller doll inside that's to be counted too*/
```

Recursive Functions

- A function is *recursive* if it calls itself.

```
int fact(int n)
{
      if(n <= 1)
          return 1;
       else
           return n * fact(n-1);
}
```

- Trace the function call

```
i = fact(3);
```

```
fact(3)
   fact(2)
        fact(1),return 1
   fact(2),return 2*1
fact(3), return 3*2
```

Recursive Functions

- Example: *power* function

```
int power(int x, int n)
{
      return n == 0 ? 1 : x * power(x,n-1)
}
```

- Trace the function call

```
x = power(5,3);
```

```
power(5,3)
   power(5,2)
        power(5,1)
             power(5,0) return 1
        power(5,1) return 5*1
   power(5,2) return 5*5
power(5,3) return 5*25
```

math.h and GCC

- math.h is not automatically linked by GCC
- To fix, explicitly link the math library when compiling by adding the -lm flag. See below.

```
#include<stdio.h>
#include<math.h>


int main()
{
    float base = 4;
    float exp = 5;
    printf("The power is %.3f\n", pow(base, exp));
    return 0;

}
```

Compilation example: gcc program.c -o output_file -lm

Practice

```
int incr(int i)
{
  static int count = 0;
  count = count + i;
  return (count);
}
main()
{
  int i,j;
  for (i = 0; i <=4; i++)
       j = incr(i);
}
```

What will be the value of j after the for loop?

To do list

- Review Chapter 8 and 9
- Do the exercises in Chapter 8 and 9
  - Partial solutions can be found in link below

http://knking.com/books/c2/answers/c8.html

http://knking.com/books/c2/answers/c9.html

Chapter 11: Pointers

Review: Variables Scope

```
int i;                  /* Declaration 1 */

void f(int i)           /* Declaration 2 */
{
  i = 1;
}

void g(void)
{
  int i = 2;            /* Declaration 3 */

  if (i > 0) {
    int i;              /* Declaration 4 */

    i = 3;
  }

  i = 4;
}

void h(void)
{
  i = 5;
}
```

Practice

In the following example, j is initialized to the same value as i, but there are two variables named i. Is this code legal? If so, what is j's initial value, 1 or 2?

```
#include <stdio.h>

int i = 1;
void f(void) {
    int j = i;
    printf("%d\n", j); // What should be the value of j?
    int i = 2;
    j = i;
    printf("%d\n", j);
}
int main()
{
    f();
    return 0;
}
```

Pointer Variable

- How do variables store in memory?
  - Main memory is divided into bytes.
  - Each byte has a unique address.
  - Each variable occupies one or more memory.
  - The address of the first byte is said to be the address of the variable.

Address	Contents
0	00110011
1	10101010
2	00000111
3	11111000
	…
n-1	10000100

Pointer Variable

- Example
  - int i = 1; /* suppose sizeof(int) = 4 */
  - Then the address of variable i is 2000

Can we use a variable to store this address?

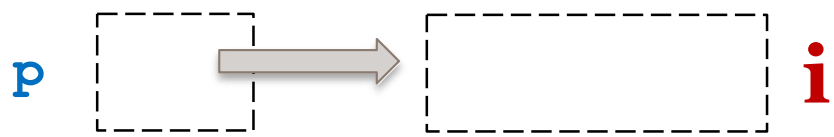
Address	Contents
	..
2000	00000001
2001	00000000
2002	00000000
2003	00000000
	...

i

Pointer Variable

- Pointer Variable
  - Store the address of a variable

Address	Contents	
	..	
2000	00000001	i
2001	00000000	
2002	00000000	
2003	00000000	
	...	
	$(2000)_{10}$	p

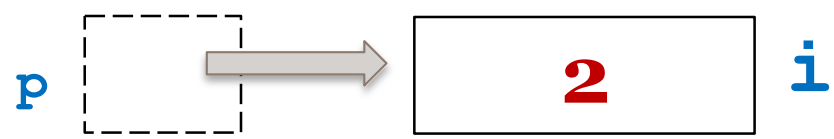


"p points to variable i"

Declaring Pointer Variables

- Pointer variable must be preceded by an asterisk.
- Every pointer variable points to **objects** of a particular type(*the reference type*).
- Examples
  - **int** * **p**; /* points only to integers */
  - **char** ***str**; /* points only to characters */
  - **double** ***q**; /* points only to doubles */
  - **int i , j , a[10], *p, *q** ; /* p and q point to integers */
- Note: There are no restrictions on what referenced type may be. In fact, a pointer variable can even point to another pointer.

The Address and Indirect Operators

- **& (address) operator** : get the address of a variable
- *** (indirection) operator** : gain the access to the object that a pointer points to
- Example:

```
int   i=2,  j,  *p;    /* p points nowhere */
p = &i;        /* p  points to integer variable i */
j = *p;  /* same as j=i  */
```